

TRAIL
OF **BITS**

Finding hidden overflows in Go: Fuzzing beyond the compiler's limits

Kevin Valerio, Security Engineer at Trail of Bits

kevin.valerio@trailofbits.com

Go's behavior creates blind spots for fuzzers

Rust build (debug)



- Panic on overflows
- Fuzzer catches crash immediately

Go build



- Silently wrap around
 - Logic errors are invisible to fuzzers
- `INT_MAX + 1 = INT_MIN`

Static analysis yields high false positives for integer overflows

A concrete example using Go vs go-panikint

```
func main() {  
    var a int8 = 120  
    var b int8 = 20  
    r := a + b  
    fmt.Printf("%d + %d = %d", a, b, r)  
}
```

With native Go

```
$ go run poc.go
```

```
120 + 20 = -116
```

Mathematically incorrect, but the program exits successfully.

A hidden logic bomb

With go-panikint

```
$ GOROOT=$pwd ./bin/go run poc.go
```

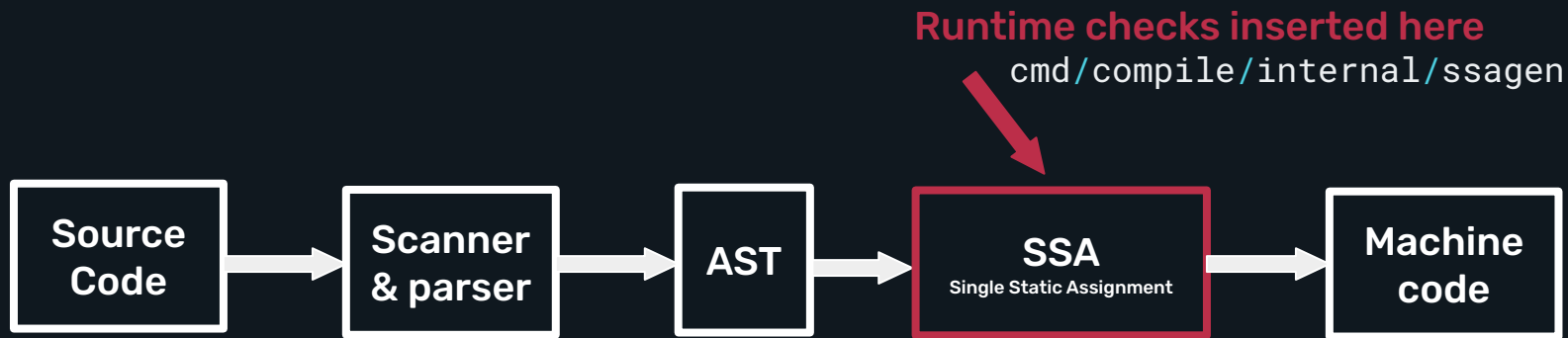
```
panic: runtime error: integer overflow in  
int8 addition operation  
goroutine 1 [running]:  
main.main()  
    ./go-panikint/poc.go:8 +0xb8  
exit status 2
```

The overflow panics, fuzzers detect it

Overview of how go-panikint was made before we go in-depth

- > Forked the Go toolchain
- > Spotted where IR handling is in the codebase
- > Modified the IR so that it adds an overflow detector after executing $+$ $-$ $/$ $*$...
- > Insert a panic if the detector don't pass

SSA was the best place to insert our checks



- > **Goal:** turning silent integer overflows into catchable runtime panics
- > **How:** patching the compiler backend during math operations
- > **Result:** every arithmetic overflow trigger runtime `.panicoverflow`

Representation of the SSA with Go vs go-panikint

Before

Native Go IR

```
v1 = OADD a b
```

After

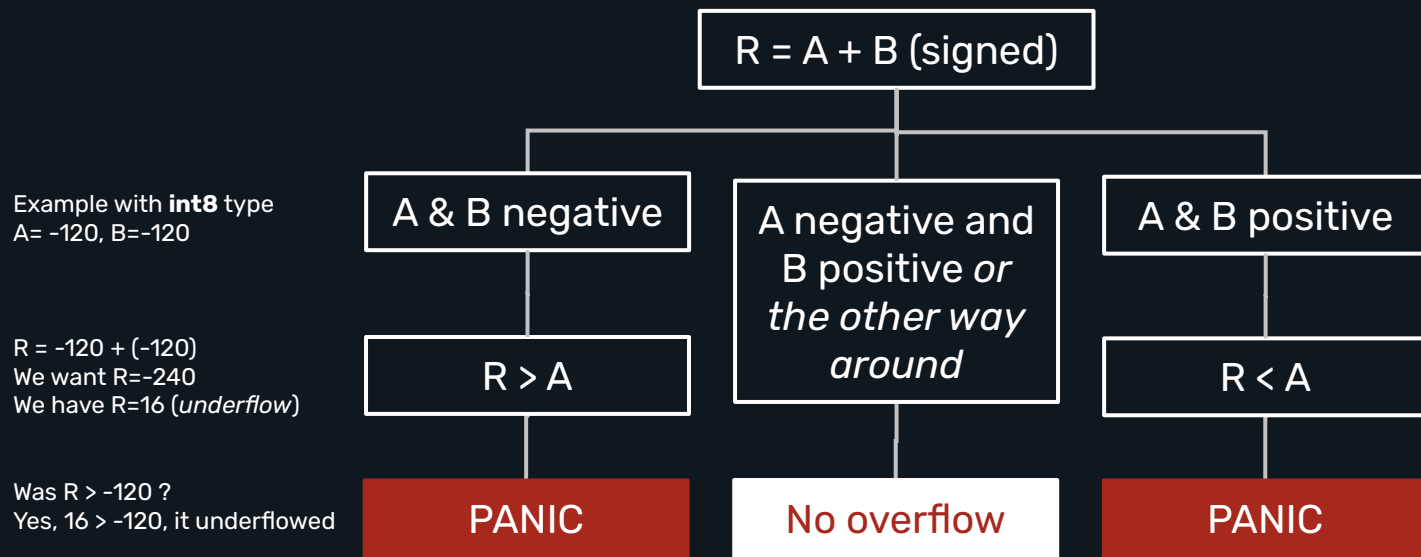
Go-Panikint IR

```
v1 = OADD a b
if (overflow_detected) {
    CALL runtime.panicoverflow()
}
```

Injected
Logic

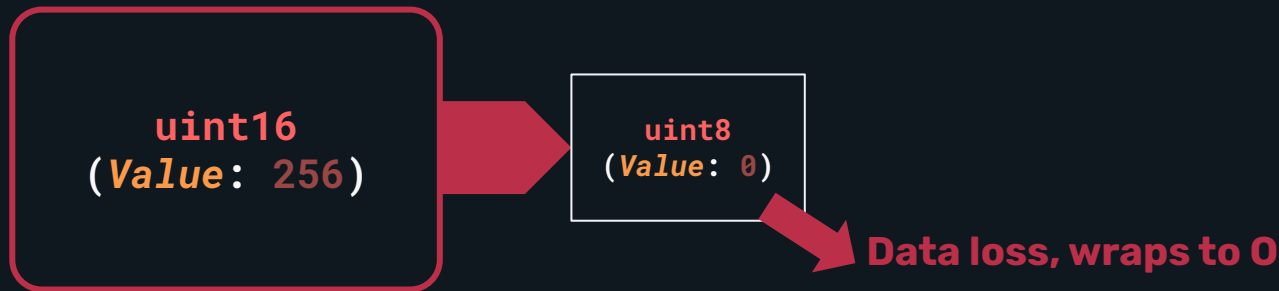
Intercepts SSA opcodes: **OADD**, **OSUB**, **OMUL**, **ODIV**
Checks are compiled directly into the binary, no external wrappers

The logic of overflows: signed addition use-case



Deterministic detection for `int8`, `int16`, `int32` + all unsigned types (`uint8`-`uint64`)
Optimized using SSA comparisons to minimize overhead

It also detect unsafe truncations



Detects unsafe type conversions (e.g., `int64` -> `int32`)
Disabled by default

A look inside decompiled checks

These runtime checks are compiled into the binary
Let's look at the decompiled check for a signed addition operation

```
if (*x_00 == '+') {  
    val = (uint32)* (undefined8 *) (puVar9 + 0x60);  
    sVar23 = val + sVar21;  
    //...  
    if ((sword) val < 0 && sVar21 < 0) && (sword) val < sVar23 || // case 1  
        ((sdword) val >= 0 && sVar21 >= 0) && sVar23 < (sdword) val){ // case 2  
        runtime.panicoverflow(); // <-- panic if overflow  
    }  
    // ...  
}
```

Checks if two
negative numbers
result in a **larger**
number

Checks if two
positive
numbers result in
a **smaller** number

Not all overflows are bugs

Some code (i.e, cryptography libs or the Go runtime) **relies** on integer wrapping

go-panikint has **two mechanisms** to handle these cases and **avoid false positives**

In-Code Comments

Any arithmetic operation can be marked as a non-issue by adding a comment

```
res := a+b // overflow_false_positive
```

Source Location Filtering

A whitelist allows us to ignore intentional overflows within specific files, such as the Go compiler's own source code or specific libraries

First trophy: Integer overflow in Cosmos SDK

```
// Paginate does pagination of all the results...
func Paginate (
    prefixStore types.KVStore,
    pageRequest *PageRequest,
    onResult func (key, value []byte) error
) (*PageResponse, error) {
    ...
    end := pageRequest.Offset +
    pageRequest.limit
    ...
}
```



fix(query): clamp offset+limit to avoid uint64 overflow (Fixes #25006) #25049

[Code](#) [Jump to bottom](#)

Open

sleeg00 wants to merge 12 commits into `cosmos:main` from `sleeg00:fix/25006-paginate-overflow`

When the sum of `offset` and `limit` parameters in a query exceeded `u64.MAX`, it would overflow.
This caused the query to return an empty list of validators instead of the expected set.

Go-panikint got integrated into gosentry

go-panikint has been **integrated** into **gosentry**, a security-oriented Go toolchain focused on fuzzing

Table of Contents

- [Build](#)
 - [Feature 1: Integer overflow and truncation issues detection](#)
 - [Feature 2: Panic on selected functions](#)
 - [Feature 3: LibAFL state-of-the-art fuzzing](#)
 - [Feature 4: Git-blame-oriented fuzzing \(experimental\)](#)
 - [Feature 5: Detect race conditions, goroutine leaks, and hangs at fuzz-time](#)
 - [Feature 6: Grammar-based fuzzing \(Nautilus\)](#)
 - [Feature 7: Generate fuzzing coverage reports from campaign](#)
- [Credits](#)

What we've seen during this talk

<https://github.com/trailofbits/gosentry>

Try it!

Using **gosentry** is straightforward. Compile it and use the generated Go binary instead of the official one. All other commands remain the same (it's a fork).

Use it whenever you're thinking of fuzzing Go apps

<https://github.com/trailofbits/gosentry>

Installation and usage

```
# Clone the repository
git clone https://github.com/trailofbits/gosentry

# Build the compiler
cd gosentry/src && ./make.bash

# Set your GOROOT and run your tests/fuzzer
export GOROOT=/path/to/gosentry

./bin/go test -fuzz=FuzzMyCode
```

Thank you!